

Project Discovery & Systems Analysis

Playbook

A practical framework for end-to-end project discovery, impact analysis, and delivery planning

Version 1.0 | Last Updated: May 2026 | Created by Zoubeir Qadi

This playbook is designed to guide business and technology teams through structured discovery. It ensures all projects are clearly defined; risks are identified early, and dependencies are understood before execution begins.

Project Name: _____

Initiative Type (Regulatory / Enhancement / New Build / Migration) :

Date : _____

Prepared By : _____

Business Unit: _____

Technology Domain: _____

Business Objective / Intake

- What are we solving?
- What is the **business problem or opportunity**?
- What triggered this request? (incident, audit finding, client demand, revenue opportunity)
- What are the **current pain points**?
- What does success look like? (KPIs, SLAs, customer outcomes)
- Is this **regulatory, revenue-driven, cost reduction, or efficiency-focused**?
- What is the **impact if no action is taken**?
- Who are the **primary users / customer segments** impacted?
- What is the **expected business value (ROI, risk reduction, growth)**?
- Is this aligned with **enterprise strategy or roadmap**?
- Are there **competing initiatives or priorities**?
- What assumptions are being made about the problem?

Scope Definition

- What are we doing vs NOT doing?

In Scope

- What **capabilities, features, and systems** are included?
- What **business processes or user journeys** are impacted?
- What data domains are included?

Out of Scope

- What is explicitly excluded?
- What is deferred to future phases?

Additional Questions:

- Are **future phases already identified**?
- Are we delivering an **MVP vs full solution**?
- What assumptions define scope boundaries?
- Are there **hidden dependencies that could expand scope**?

Stakeholders & Ownership

- Who is involved?
 - Business Owner
 - Product Owner
 - Technology Lead / Architect
 - Delivery Manager / Scrum Master
 - Upstream System Owners
 - Downstream Consumers
 - Vendors / Third Parties
 - Risk / Compliance / Audit
- Who has **decision-making authority**?
- Who approves **requirements, design, and release**?
- Who owns **production support**?
- Are there **cross-functional or external stakeholders**?

System / Platform Identification

- What systems are touched?
 - List all applications involved
 - Internal vs External systems
 - Legacy vs Modern platforms
 - Vendor / third-party systems
- Are **new systems being introduced**?
- Are any systems being **retired or replaced**?
- What are the **system roles** (source, processor, consumer)?
- Any **vendor SLAs or constraints**?

System Impact Assessment

- How are systems impacted?

For each system:

- Type of change: **New / Enhancement / No Change**
- Layer impacted:
 - UI
 - API
 - Middleware
 - Backend services
 - Data layer
- Is there **cross-platform impact** (web, mobile, backend)?
- Are **multiple teams required**?
- Will this impact **performance, scalability, or availability**?
- Are there **backward compatibility concerns**?
- Will existing functionality be **deprecated or modified**?

Upstream Impact (Inputs)

- What feeds into us?
 - Source systems providing data
 - APIs, batch jobs, file transfers, streams
- Will upstream systems need to **change schemas or interfaces**?
- Are there **data quality or latency concerns**?
- Are there **SLAs or timing dependencies**?
- What happens if upstream **fails or sends bad data**?
- Are there **environment dependencies (test vs prod alignment)**?

Downstream Impact (Outputs)

- Who depends on us?
 - Systems consuming outputs
 - Reporting, analytics, dashboards
 - External clients or partners
- Will downstream systems **break if we change structure or timing**?
- Are **data contracts impacted**?
- Do we need **consumer validation or certification testing**?
- Are there **regulatory or reporting impacts**?
- Do we need to **communicate changes proactively**?

Data Impact Assessment

What happens to the data?

- New data elements introduced
- Changes to schema / data model
- Data storage, retention, archival
- Is there **PII, financial, or sensitive data**?
- Are **encryption, masking, or tokenization** required?
- Is **data migration or backfill** required?
- What are **validation, transformation, and enrichment rules**?
- Are there **data lineage and audit requirements**?
- Will this impact **data reconciliation processes**?

Integration & Interfaces

- How do systems talk?
 - APIs (REST, SOAP)
 - File transfers (SFTP, batch)
 - Messaging (Kafka, MQ)

- Are integrations **real-time, near real-time, or batch**?
- Are there **API contract changes**?
- Any **authentication/authorization updates** (OAuth, SSO, tokens)?
- Are there **rate limits or vendor constraints**?
- What are **failure handling and retry mechanisms**?

Process / Workflow Impact

- What changes operationally?
 - User journey changes
 - Operational workflows
 - Manual vs automated processes

- Are new workflows introduced?
- Are existing workflows modified or removed?
- Are there **new approvals or controls**?
- Will users require **training or documentation**?
- Will this impact **SOPs or SLAs**?

Assumptions & Dependencies

- What are we assuming?

Assumptions

- Scope, timeline, technical feasibility

Dependencies

- External teams
 - Vendors
 - Regulatory timelines
 - Infrastructure readiness
-
- Which assumptions are **high risk**?
 - What dependencies could **block delivery**?
 - Are dependencies **tracked and owned**?

Risks & Constraints

- What could go wrong?

Technical Risks

- Integration failures
- Data inconsistency
- Performance degradation

Business Risks

- Missed deadlines

- Regulatory non-compliance
 - Customer impact
-
- What is the **likelihood vs impact**?
 - What are **mitigation strategies**?
 - Are there **resource or budget constraints**?
 - Any **single points of failure**?

Test Strategy

- How will we validate?
 - Unit Testing
 - Integration Testing
 - System Testing
 - UAT
 - Performance / Load Testing
- Who owns each testing phase?
- Are **test environments available and stable**?
- Is **test data available and compliant**?
- Are **automation frameworks used**?

- What is the **entry/exit criteria** for each phase?

T-Shirt Sizing / Complexity

- How big is the project/effort?
 - XS / S / M / L / XL
- Is this **single-system** or **multi-system**?
- Are there **unknowns increasing complexity**?
- Are integrations **simple** or **highly coupled**?
- Any **new technology** or **learning curve** involved?

➤ Milestones & Timeline

- What are the phases?
 - Requirements
 - Design
 - Development
 - Testing
 - Release
- Are there **fixed deadlines** (regulatory, client)?
- Is there a **phased rollout** or **pilot**?
- What are **critical path items**?
- Are there **release windows** or **blackout periods**?

Tech Scope of Work

- What are we delivering?
 - APIs
 - UI changes
 - Backend services
 - Data pipelines
 - Infrastructure updates

- What exactly will be **built, modified, or removed**?
- What is **explicitly out of scope**?
- Are there **non-functional requirements** (performance, security, scalability)?

Engagement Model

- How will we be involved end-to-end?
 - Intake
 - Design
 - Build
 - Testing
 - Deployment

- Who owns each phase?
- Where are the **handoffs**?

- Is delivery **Agile, Waterfall, or Hybrid**?
- How are **issues, risks, and changes** tracked?

Open Questions / Gaps

- What don't we know yet?
 - Missing requirements
 - Unclear dependencies
 - Undefined architecture
- What is blocking **estimation or design**?
- What needs **clarification before moving forward**?

Decisions Needed

- What do we need now?
 - Approvals (business, tech, risk)
 - Architecture decisions
 - Scope confirmation
 - Funding / resource allocation
- What decisions are **time-sensitive**?
- Who is accountable for each decision?
- What are the **next immediate steps**?